Informe de pràctica final: Robòtica mòbil

Irene Miralles
Martí Andreu

Índex

1. Idea inicial 3
2. Comportament general 3
    2.1. Manual d'ús 4
3. Explicació del codi 4

1. Idea inicial

El punt de partida de la pràctica era programar un robot per a que es mogués amb una sèrie de requisits bàsics que englobaven el temari vist a les sessions. Els requisits inicials eren els següents:

- El robot havia de poder desplaçar-se per l'entorn controlant tant la velocitat lineal com la rotatòria.

- El robot havia d'utilitzar les dades d'el sensor làser.

- El robot havia de publicar al topic *command vel.*

- S'havia de crear un node propi, on gestionar les diferents subscripcions, publicacions i desenvolupar la lògica del moviment.

- El robot havia de fer ús de la odometria per a poder conèixer la seva posició i orientació.

A partir d'aquests requisits bàsics, vam dissenyar la nostra idea. Aquesta consistia en un robot que, un cop alertat, fos capaç de detectar una persona intrusa, seguir-la amb precaució i actuar sota certes comandes.

Aquest enfocament ens permetia combinar la odometria amb la informació del làser.

2. Comportament general

El robot comença sempre inactiu, en estat d'espera, i no inicia cap acció fins que rep un senyal que indica la presència d'un intrús. Un cop activat, el robot entra en mode seguiment. La seva missió principal és detectar l'intrús i seguir-lo mentre manté una distància de seguretat, per tal d'evitar xocar.

Durant el seguiment, el robot es mou segons la distància a l'intrús. Si aquest s'allunya massa, avança per acostar-s'hi; si s'apropa massa, retrocedeix per mantenir l'espai. Per tal d'evitar col·lisions, el robot s'atura si detecta obstacles massa a prop darrere seu mentre retrocedeix.

Quan ha seguit un intrús durant set metres, el robot s'atura i realitza una volta de 360 graus sobre si mateix. Aquesta maniobra pretén emular una acció de supervisió. Tanmateix, cal destacar que durant aquesta acció de gir, el robot no utilitza el làser. Per tant, en aquest moment no detecta persones ni obstacles. Utilitza la odometria tant per mesurar l'angle girat com per calcular la distància recorreguda.

Cal comentar també que en qualsevol moment, es pot enviar una ordre perquè el robot giri 360 graus (per fer una "inspecció" puntual) o 180 graus (en cas que estigui encarat cap a

una paret o hagi perdut el rastre de l'intrús). El gir de 180 graus permet al robot reorientar-se i reiniciar la cerca des d'una nova direcció, l'ajuda a trobar l'intrús perdut.

2.1. Manual d'ús

Com ja ha estat esmentat el comportament del robot es controla mitjançant la publicació de missatges als topics corresponents.

Per activar el seguiment d'un intrús, cal publicar un missatge de tipus std_msgs/Bool amb el valor true al topic *intruder_command*. AIxò es fa mitjançant la següent comanda:

rostopic pub /intruder_command std_msgs/Bool "data: true"

Si el que es vol es activar el gir automàtic de 360 graus és necessari tornar a enviar el mateix missatge, amb la mateixa comanda.

Per indicar al robot que faci una mitja volta manualment (per exemple, si ha perdut l'intrús o es troba davant d'un obstacle fix), s'utilitza la mateixa comanda però en el topic *manual_turn.*

3. Explicació del codi

El comportament del robot es basa en una sèrie de modes de funcionament, controlats per la variable global *mode*. D'aquesta forma es gestionen els diferents estats del robot, cadascun amb un comportament específic:

- **Mode 0 – Espera:** El robot està aturat i no actua fins que rep una ordre externa (publicada al topic).

- **Mode 1 – Seguiment:** El robot utilitza el làser per detectar un intrús i seguir-lo, i simultàniament usa l'odometria per calcular la distància recorreguda.

- **Mode 2 – Gir de 360°:** Quan ha recorregut més de 7 metres, el robot gira 360 graus sobre si mateix. També gira quan se li ordena.

- **Mode 3 – Gir de 180°:** El robot rep una ordre externa per reorientar-se i gira mitja volta. És útil quan ha perdut l'intrús o està encarat cap a una paret.

El codi està dividit en diferents funcions, on es controlen els modes i el comportament. En total són cinc funcions: *ScanCallback, IntruderCallback, manualTurnCallback, odomCallback, main.*

Funció *scanCallback*

La funció *scanCallback* només actua en mode 1 i controla el comportament del robot quan aquest està seguint l'intrús.

Es defineix una distància mínima i una variable *index_intrus*, que es fa servir per guardar la posició angular (índex del làser) de l'objecte més proper detectat, l'intrús. Aquestes variables s'inicialitzen amb valors irreals: 4, sabent que el làser del turtlebot arriba fins a 3.5 metres i -1 com a índex.

Seguidament, s'itera amb un rang de 45 graus, que em considerat com a visió frontal del robot. Si dins d'aquest rang es troba un objecte a una distància superior a 0.2 m (per descartar possibles errors) i inferior a la distància mínima registrada fins al moment, es considera com a candidat a ser l'intrús, i s'actualitza la variable *min_dist*. En definitiva, el robot seguirà a l'objecte que detecti com a més pròxim.

Si després d'aquesta cerca no s'ha detectat cap objecte dins del rang especificat, s'interpreta que no hi ha intrús, i el robot s'atura fixant les velocitats lineal i angular a zero.

En canvi, si s'ha detectat un intrús, s'imprimeix la seva distància i es decideix el comportament de desplaçament del robot. Si l'intrús es troba a més de 0.6 metres, el robot avança. Si està a menys de 0.5 metres, retrocedeix per mantenir una distància de seguretat. Si es troba dins d'aquest marge, el robot manté la posició.

```
if (min_dist > 0.6) {

        velocitat.linear.x = 0.2;

    } else if (min_dist < 0.5) {

        velocitat.linear.x = -0.2;

    } else {

        velocitat.linear.x = 0.0;

    }
```

Per evitar col·lisions durant el retrocés, es controla un rang de cinc graus darrere el robot, i si es detecta un obstacle molt proper mentre el robot retrocedeix, el robot s'atura. La condició de parada és la següent:

```
f (msg->ranges[i] > 0.01 && msg->ranges[i] < 0.2 && min_dist < 0.5) {
```

Vam afegir el *>0.01* ja que el robot es parava contínuament (encara que no hi hagués cap objecte darrere), vam pensar que una possible causa podria ser mesures falses que

enviessin una distància de 0. Desconeixem si aquesta era realment la causa però en afegir això va disminuir el problema.

Finalment, la funció ajusta la velocitat angular del robot per centrar-lo respecte a la posició de l'intrús. Si aquest es troba lleugerament cap a l'esquerra (índexs 0–22), el robot gira cap a l'esquerra. Si es troba cap a la dreta (índexs 338–359), gira cap a la dreta. Aquest ajust permet mantenir l'intrús aproximadament centrat davant del robot.

Funció intruderCallback

La funció *intruderCallback* s'encarrega de gestionar les ordres externes rebudes pel topic */intruder_command*, que es publicaran manualment mitjançant la comanda ja esmentada. Aquesta funció permet avançar seqüencialment entre modes: de mode 0 a 1, i de mode 1 a 2. Quan es rep un missatge amb valor true.

Si el robot es troba en mode 0 (espera), canvia al mode 1 (seguiment).

Si ja es troba en mode 1, es canvia al mode 2 (gir de 360 graus). En aquest moment s'inicialitzen les variables relacionades amb el gir: *previous_yaw* (orientació en el moment d'inici, moment anterior) i angle_acumulat, que servirà per comptar l'angle total recorregut durant el gir.

Si el robot ja està en mode 2, s'ignora la nova ordre, ja que el robot ja es troba en procés de gir.

Funcio *manualTurnCallback*

La funció *manualTurnCallback* actua de manera molt similar al mode 2 de l'anterior funció però està destinada a controlar el gir de 180 graus (mode 3). Aquest també s'inicia per una ordre externa, publicada al topic */manual_turn*. Quan rep el senyal, el robot canvia al mode 3, atura qualsevol moviment que pogués estar fent, i torna a inicialitzar les variables *previous_yaw* i *angle_acumulat* per preparar el càlcul del gir. Aquesta volta s'utilitza, per exemple, quan el robot ha perdut l'intrús o s'ha encarat cap a un mur, i cal que es reorienti cap a una nova direcció per continuar buscant.

Aquesta va ser la última funció que vam definir. La vam pensar i incloure quan el projecte ja estava finalitzat. La idea va sorgir del fet que a vegades, quan provàvem el robot real, aquest es confonia i s'encarava amb una paret, aleshores vam decidir tenir alguna forma de comunicar-li al robot que deixés d'enfocar la paret.

Funció *odomCallback*

La funció *odomCallback* s'encarrega de la part del codi que fa ús de la odometria, és a dir, la distància recorreguda i el gir. Per tant, actua en més d'un mode.

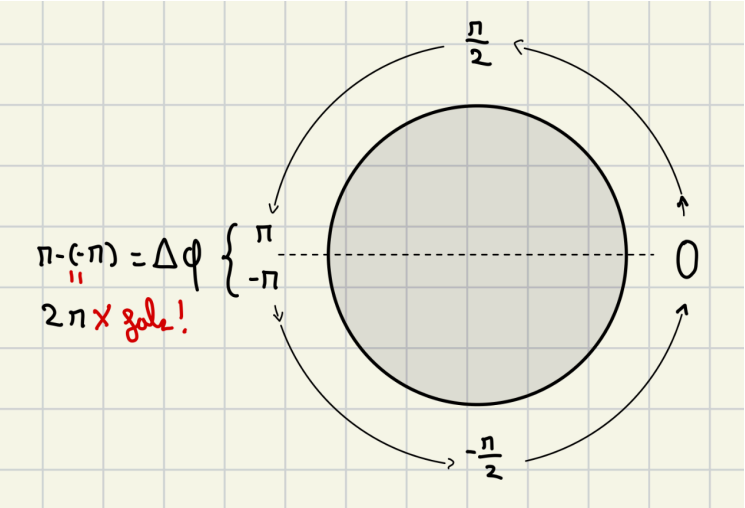
Quan el robot es troba en mode 1, es calcula la distància recorreguda acumulada. Aquest càlcul es fa a partir de la fórmula de la distància entre dos punts a l'espai. Aquesta distància es va sumant i s'imprimeix a pantalla per facilitar el seguiment. Si es detecta que s'han recorregut més de 7 metres, es canvia a mode 2 (gir automàtic) i es reinicien les variables de gir, tal com s'ha explicat anteriorment.

```
double delta = sqrt(pow(current_pose.x - previous_x, 2) +
pow(current_pose.y - previous_y, 2));

dist_recorreguda += delta;
```

Quan el robot està en mode 2 o 3, la funció s'encarrega de controlar el gir. Es calcula la diferència d'angle entre la posició actual (yaw) i l'anterior (previous_yaw).

Aquesta va ser la part del codi que més problemes va donar. Els angles es representen en radians dins d'un rang circular que va de –π a +π. Això vol dir que, per exemple, un angle molt proper a –π (com –3.13) és gairebé idèntic a un angle molt proper a +π (com 3.14), però numèricament semblen molt separats. Això provocava errors quan en calcular la diferència entre dos angles.



Per exemple, imaginem que el robot gira amb una orientació propera a +π (per exemple, 3.1 radians), i en la següent lectura l'angle és a –3.1. Si simplement es fa la resta directa ($-3.1 - 3.1$), s'obté una diferència de –6.2 radians, quasi una volta sencera, quan en realitat el robot només ha girat 0.08 radians en sentit horari.

Per resoldre aquest problema vam implementar les següents línies de codi:

```
if (delta_angle > M_PI) delta_angle -= 2 * M_PI;
```

```
if (delta_angle < -M_PI) delta_angle += 2 * M_PI;
```

D'aquesta forma, si la diferència d'angle era major que pi restem 2π per portar-la al rang correcte i al revés.

Durant el gir, el robot manté la velocitat lineal a zero i gira amb una velocitat angular constant. El valor màxim de gir (*limit_angle*) es defineix segons el mode: 2π radians per al gir complet (mode 2), o π radians per a la mitja volta (mode 3).

Un cop s'ha completat el gir, el robot torna automàticament al mode 1, reinicia les coordenades inicials i atura la rotació.

Funció main

Finalment, la funció main és la que inicialitza el node de ROS (*seguiment_intrus*). Aquí es configuren totes les subscripcions als diferents topics: */scan* per al làser, */odom* per a l'odometria, */intruder_command* per a l'activació de modes, i */manual_turn* per a l'ordre de gir manual.

També es configura el publisher al topic */cmd_vel* i s'inicialitzen per primer cop les variables *previous_x* i *previous_y* per garantir que, en començar el seguiment, ja es disposi d'una posició inicial vàlida per calcular la distància recorreguda.